

Hardware Inventory - Functional Specification Document

Functional Specification Document (FSD)

Product: Hardware Inventory System
Document type: Functional Specification (design of behavior)
Audience: Developers (primary); owners/managers for flows and business rules
Version: 1.0
Date: 2026-07-24
Related docs: [FRD.md](#) (requirements), [USER_GUIDE.md](#), [packaging/GO-LIVE-NOTES.md](#), [packaging/README.md](#)

This FSD describes **how** the as-built system fulfills the FRD: architecture, flows, business rules, UI surfaces, and API surface. It is not a pixel-level UI kit or full OpenAPI spec.

1. Document control

Item	Value
Requirements source	docs/FRD.md
Implementation truth	prisma/schema.prisma, src/, frontend/src/
Auth note	Live model is JWT + RBAC . Ignore any older “no JWT” notes in AI context docs.

2. System context

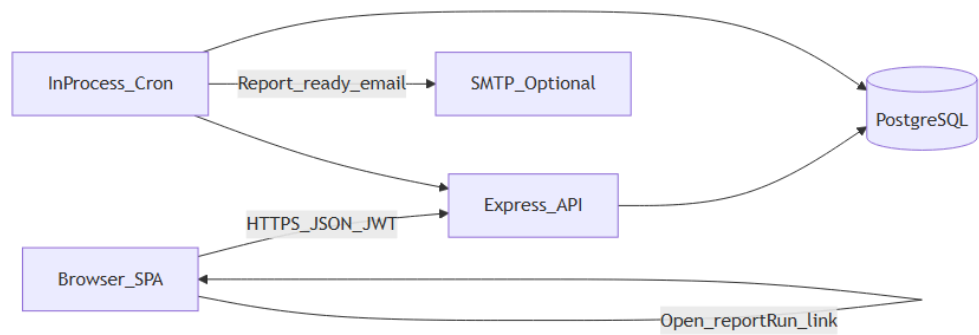


Diagram 1

Layer	Technology
Frontend	React 19, TypeScript, Vite 6; inline styles; no UI kit
Backend	Node ≥ 20 , Express 5, Zod validation, tsx
Database	PostgreSQL via Prisma 7 (@prisma/adapters-pg)
Auth	Username/password $\rightarrow$ JWT Bearer; bcrypt password hashes
Jobs	node-cron in API process (Asia/Kolkata)
Email	Optional Nodemailer SMTP for scheduled reports

3. Architecture overview

- **buildApp()** (src/app.ts) mounts /api/login, /api/health, then JWT actingUserMiddleware on /api/*, then domain routers.
 - Production may set SERVE_FRONTEND=1 so Express serves frontend/dist and SPA fallback for non-API GETs.
 - Validation schemas live in src/validation/schemas.ts.
 - Inventory/pricing/report logic lives under src/services/ and src/lib/.
 - Frontend API client: frontend/src/api/client.ts (Authorization + optional X-Acting-User-Id).
-

4. Auth and RBAC

FR: FR-AUTH-*, role matrix §7 in FRD.

4.1 Login flow

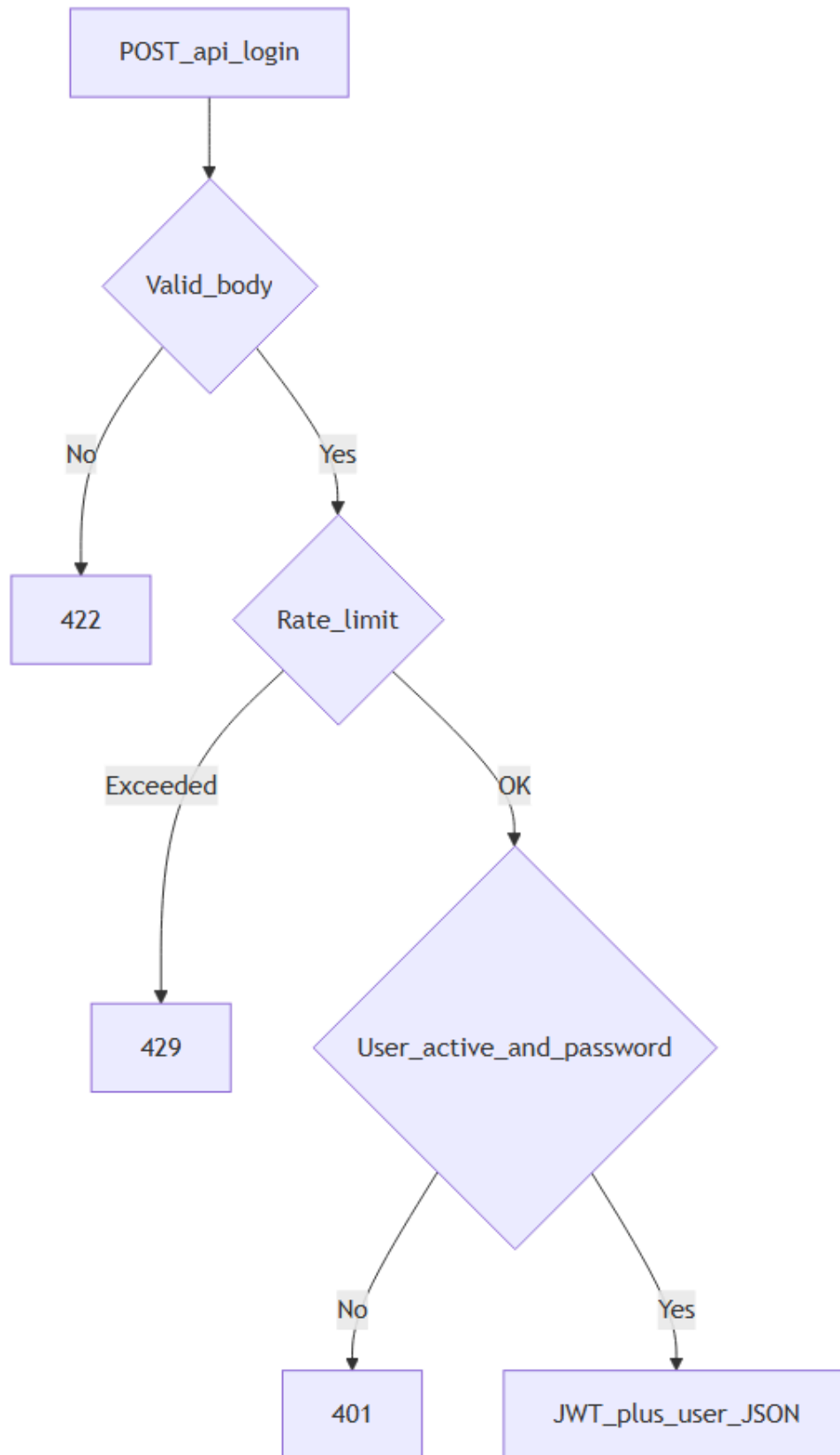


Diagram 2

- Route: POST /api/login (no Bearer required).
- Success: access token + user profile; client stores token (localStorage; ~7-day JWT lifetime per env/docs).

- Rate limit: LOGIN_RATE_LIMIT_MAX / LOGIN_RATE_LIMIT_WINDOW_MS (default 10 / 15 min).

4.2 Acting user

- Middleware: src/middleware/actingUser.ts.
- Requires Authorization: Bearer <token>.
- Optional header X-Acting-User-Id: only if JWT user has admin access (ADMIN or MANAGER); switches req.actingUser.
- hasAdminAccess(role) treats MANAGER as admin for compatibility.
- requireAdmin middleware gates back-office routes.

4.3 Session API

- GET /api/session — current JWT user, acting user, and (for admin) list of users available for acting-as switcher.

5. Information architecture

Source: frontend/src/Sidebar.tsx, frontend/src/App.tsx, frontend/src/featureFlags.ts.

Tab id	Label	Component	Visibility
home	Home	HomeView	All
pos	Sell (POS)	POSView (in App)	All
outstanding	Outstanding	OutstandingView	All
invoices	Invoices	ReprintInvoicePage	All
reporting	Reporting	ReportingPage (+ SavedReportViewer)	Admin + FEATURE_FLAGS.reporting
products	Products	ProductsPage	All (mutations admin only)
promotion	Promotions	PromotionsPage	Admin + catalogPromotions (default off)
inventory	Overview	InventoryView	All
purchase	Purchases	PurchaseView	Admin
adjustment	Adjustments	AdjustmentView	Admin
settings	Settings	SettingsPage	All

Unauthenticated → LoginPage. Soft redirect to Home if a cashier hits admin-only tabs. Deep link ?reportRun=<id> opens Reporting for admin when reporting is enabled.

Current feature flags: reporting: true; catalogPromotions: false; other catalog/inventory sub-flags false.

6. Module functional design

6.1 Home

FR: FR-HOME-*

Aspect	Design
Goals	Orient staff; jump to POS/inventory; show alerts and (admin) revenue
UI	Shortcuts, recent activity, stock alert counts, admin charts via report timeseries APIs
APIs used	Sales recent-activity; report dashboard/revenue series (admin); products for alert counts
Edges	Cashier UI emphasizes sell path; charts may be empty on fresh installs

6.2 Catalog (products)

FR: FR-CAT-*

Business rules

- Product has base unit + optional additional ProductUnit rows with conversionToBase.
- Barcode.code unique; types PRODUCT / PACK / INTERNAL / SUPPLIER.
- brandCode unique case-insensitively when non-blank; blank may auto-generate BC-#####.
- Stock on hand: Product.currentStock (base units); ledger via StockMovement.
- **Price review:** cheaper purchase → keep sellingPrice, set priceReviewNeeded, suggestedSellingPrice, priceReviewNote. Cost increase → selling may rise via markup (src/lib/productPricing.ts / purchase receive path). Admin apply/dismiss per product or all.
- GST fields: hsnCode, cgstPercent / sgstPercent / igstPercent.
- Status ACTIVE / INACTIVE.

UI surfaces

- Products grid: paginated (50), server search; some columns hidden in grid but present in forms/import.
- Add/edit, batch entry, CSV/Excel import ($\leq \sim 1500$), download template.
- Price review banner: Apply / Keep current (per item and bulk).

API (/api/products)

Method	Path	Auth
GET	/	Authenticated (list/search/paging)
GET	/:id/stock	Authenticated
POST	/batch	Admin
PATCH	/:id	Admin
DELETE	/:id	Admin
POST	/price-review/apply-all	Admin
POST	/price-review/dismiss-all	Admin
POST	/:id/price-review/apply	Admin
POST	/:id/price-review/dismiss	Admin

Edges

- Delete may fail if product is referenced by historical lines (constraint/error handling as implemented).
- Bulk create uses chunked transactions (~100/chunk).

6.3 Parties (suppliers and customers)

FR: FR-PARTY-*

Entity	API	Notes
Supplier	GET/POST /api/suppliers — admin router	Unique name; used by purchases
Customer	GET /, GET /:id, POST / under /api/customers — authenticated	Optional partyGstNo, partyState; phone unique when set

UI: supplier management primarily on Purchases screen; customers used from POS / outstanding / reports.

6.4 Purchases

FR: FR-PUR-*

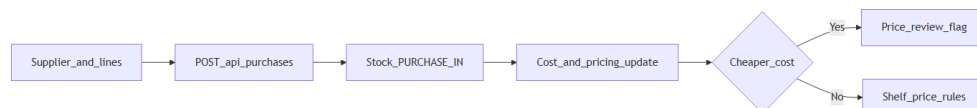


Diagram 3

Business rules

- Status enum: DRAFT / RECEIVED / CANCELLED (create path records received stock as implemented in purchase service).
- Unique purchaseNumber; rare collisions retried.
- Lines: quantity in selected unit → quantityInBase; unit cost drives catalog cost / avg cost / selling rules.
- Payments: POST /api/purchases/:id/payments updates paidAmount / balanceAmount.
- Import: match by brand code and/or product name; optional single supplier column; >40 lines → summary UI before Record Purchase.

API (/api/purchases) — all admin

Method	Path
GET	/
POST	/
GET	/:id
POST	/:id/payments

Edges

- Cashier: 403 / nav hidden.
- Large imports need JSON_BODY_LIMIT (default 15mb).

6.5 Point of sale (POS)

FR: FR-POS-*

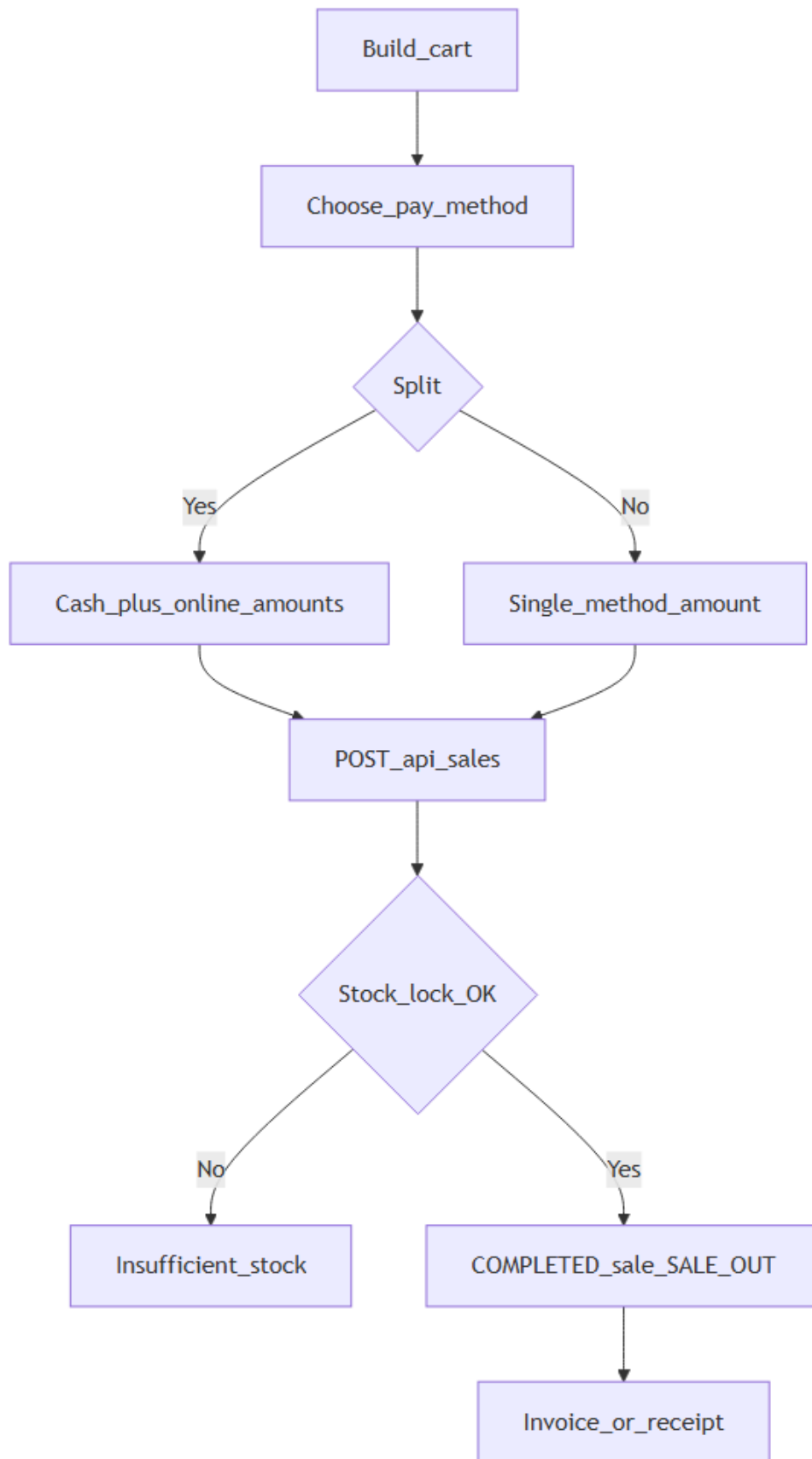


Diagram 4

Business rules

- Sale statuses: DRAFT / COMPLETED / CANCELLED / RETURNED (POS complete path produces completed sales with movements).
- Payments: SalePaymentMethod CASH | ONLINE_BANKING; multiple payments allowed over time.
- Split: cash + online entered separately; **remainder = balance due** (not auto-assigned).
- Totals: subtotal, discount, tax, transportAmount, total, paid, balance.
- Customer: walk-in fields and/or customerId; snapshots for name/GST/state on the sale.
- Document numbers: BIL- $\{n\}$ vs INV- $\{n\}$ from `src/lib/documentNumbers.ts`; starts via SALE_NUMBER_BILL_START / SALE_NUMBER_TAX_START.
- Concurrent completion uses stock locking so oversell is rejected.

UI

- Product search/cart, discounts, payment panel, processing indicator on long confirms, invoice print integration (`frontend/src/invoice/`).

API (`/api/sales`) — authenticated (no admin gate)

Method	Path
POST	<code>/</code> (create/complete sale as validated)
GET	<code>/recent-activity</code>
GET	<code>/outstanding</code>
GET	<code>/search</code>
GET	<code>/by-number/:saleNumber</code>
GET	<code>/:id</code>
POST	<code>/:id/payments</code>

Edges

- Insufficient stock → error to client; cart may be retried with reduced qty.
- Body `createdById` must match acting user where enforced.

6.6 Outstanding

FR: FR-OUT-*

Aspect	Design
UI	<code>OutstandingView</code> — list sales with <code>balanceAmount > 0</code> ; record payment
API	GET <code>/api/sales/outstanding</code> ; POST <code>/api/sales/:id/payments</code>

Aspect	Design
Rules	Payments append SalePayment rows; update paid/balance

6.7 Invoices (reprint)

FR: FR-INV-*

Aspect	Design
UI	ReprintInvoicePage — search/lookup by sale number; print layout from invoice module
API	GET /api/sales/search, GET /api/sales/by-number/:saleNumber, GET /api/sales/:id
Branding	UI strings: frontend/src/lib/branding.ts (VITE_COMPANY_NAME); invoice letterhead fields in frontend/src/invoice/invoiceBranding.ts

6.8 Stock overview and adjustments

FR: FR-STK-*

Aspect	Design
Overview UI	InventoryView — levels, search/sort, in/low/out status vs reorder
Adjustment UI	AdjustmentView — product, target/delta as implemented, reason, note
API	Adjust: POST /api/stock-adjustments (admin). Stock read also via products.
Rules	StockAdjustment stores before/after/difference; movements ADJUSTMENT_IN/OUT, DAMAGE_OUT, etc. as typed
Edges	Cashiers see overview only

6.9 On-demand reporting

FR: FR-RPT-*

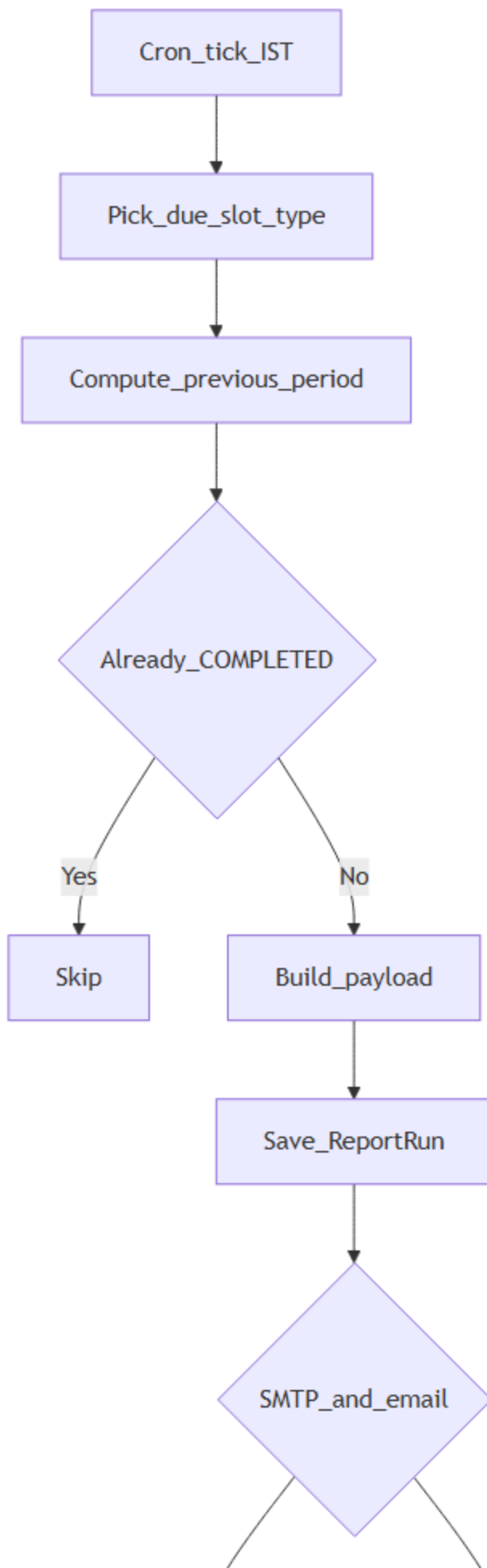
All under /api/reports, **admin only**.

Method	Path	Query
GET	/sales-summary	from, to (YYYY-MM-DD)
GET	/tax-invoice-sales	from, to
GET	/sales-by-product	from, to
GET	/sales-by-customer	from, to
GET	/supplier-payments	from, to
GET	/purchases	from, to
GET	/gross-margin	from, to
GET	/dashboard-timeseries	days (default 14)
GET	/sales-revenue-series	granularity day week month; buckets

UI: ReportingPage — pick report and date range; charts for series endpoints. Indexes on sale/purchase createdAt support range queries.

6.10 Scheduled owner reports

FR: FR-SCH-*



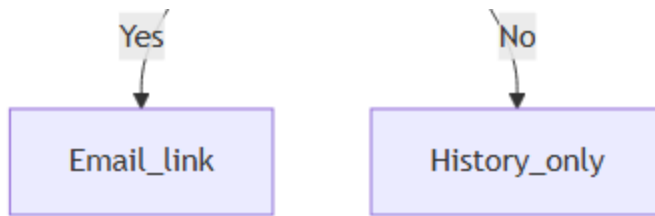


Diagram 5

Configuration

Singleton ReportScheduleConfig id=default:

- notifyEmail (required if any type enabled)
- Per type: enabled + period (DAILY | MONTHLY | QUARTERLY | YEARLY)
- Types: Sales, Inventory, Supplier outstanding, Customer outstanding

Payload content

Type	Content
SALES	Sales summary + gross margin for period
INVENTORY	Active product stock snapshot (value, OOS, below reorder)
SUPPLIER_OUTSTANDING	RECEIVED purchases with balance > 0, by supplier
CUSTOMER_OUTSTANDING	COMPLETED sales with balance > 0

Periods generate the **previous** completed window. Quarterly uses Indian FY (Q1 Apr–Jun ... Q4 Jan–Mar).

Scheduler

- Files: src/jobs/reportScheduler.ts, src/services/scheduledReports.ts, src/lib/reportScheduleSlots.ts
- Enable: REPORT_SCHEDULER_ENABLED=1|true, or default **on** when NODE_ENV=production
- Cron: REPORT_CRON_IST default 0,30 6-8 * * * Asia/Kolkata
- Stagger ~30 min: Sales → Inventory → Supplier → Customer; **one type per tick**
- Unique (reportType, period, periodStart); retries failed/incomplete; skips completed

Email

- Requires notifyEmail + SMTP_HOST (and related SMTP_* vars)
- Link: {APP_PUBLIC_URL}/?reportRun={id}
- Without SMTP: run still COMPLETED in DB

API (/api/scheduled-reports) — admin

Method	Path
GET/PUT	/config
GET	/runs?limit=
GET	/runs/:id

UI: Reporting → Scheduled owner reports; history; SavedReportViewer for a run.

6.11 Promotions (optional)

FR: FR-PROMO-01

- Schema: Promotion + PromotionProduct; scope CART / PRODUCT / CATEGORY; percentage discount; optional date window.
 - API: GET /api/promotions (auth); POST/PUT/DELETE admin.
 - UI: hidden unless FEATURE_FLAGS.catalogPromotions === true (default false).
-

6.12 Settings

FR: FR-SET-*

- SettingsPage: Light / Dark / System via useTheme — **browser local only**; no settings API.
-

7. Cross-cutting concerns

Concern	Design
Validation	Zod schemas in src/validation/schemas.ts; 422 with field details on failure
Errors	Central errorHandler; domain errors return JSON { error } (+ details)
CORS	buildCorsOptions(); production uses CORS_ORIGIN list
Body size	JSON_BODY_LIMIT default 15mb for bulk imports
Feature flags	frontend/src/featureFlags.ts — compile-time constants
Document numbers	Env start values; separate bill/tax series
Branding	VITE_COMPANY_NAME / short name; invoice static letterhead module

Concern	Design
Acting user audit	Sale/purchase/adjustment creator IDs must align with acting user checks

8. Data model summary

Primary entities (see `prisma/schema.prisma` for full fields):

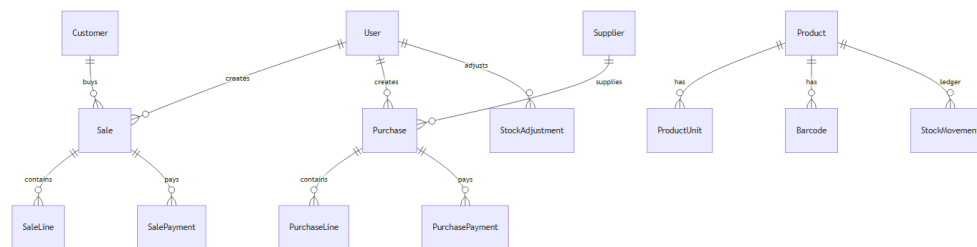


Diagram 6

Entity	Role
User	Staff; role ADMIN/MANAGER/CASHIER; passwordHash
Product	Catalog + currentStock, costs, MRP, selling, GST, price review
ProductUnit / Barcode	Sell/buy units and scan codes
Supplier / Customer	Parties
Purchase / PurchaseLine / PurchasePayment	Stock in + payables
Sale / SaleLine / SalePayment	Stock out + receivables
StockMovement	Immutable-style ledger of qty changes
StockAdjustment	Explicit correction audit
Promotion*	Optional discounts
ReportScheduleConfig / ReportRun	Owner schedule + saved payloads

9. Deployment modes (functional view)

Mode	Behavior
Dev API + Vite	API (e.g. :4000) + frontend (:5173); CORS allows local Vite origins
<code>npm run start:prod</code>	Built SPA served from Express when <code>SERVE_FRONTEND=1</code> / production dist present
Railway / hosted	See <code>packaging/RAILWAY.md</code> ; same API + DB + env
Backups	<code>npm run db:backup / restore</code> — <code>scripts/BACKUP.md</code>

Mode	Behavior
Health	GET /api/health unauthenticated

Install steps are **not** duplicated here; see packaging docs.

10. FR → FSD index

FR ID prefix	FSD
FR-AUTH	§4
FR-HOME	§6.1
FR-CAT	§6.2
FR-PARTY	§6.3
FR-PUR	§6.4
FR-POS	§6.5
FR-OUT	§6.6
FR-INV	§6.7
FR-STK	§6.8
FR-RPT	§6.9
FR-SCH	§6.10
FR-PROMO	§6.11
FR-SET	§6.12
NFR-SEC / CON / PERF / OPS / OBS / I18N	§4, §7, §9

11. Known optional / disabled surfaces

Documented so requirements are not mistaken for missing defects:

Item	Status
Promotions nav/UI	Flag off by default
Catalog brands / product types / price books nav	Flags off
Inventory order / receive / supplier returns / counts nav	Flags off
Invoice company address/GST/bank	Code constants, not admin UI
Server-side user admin CRUD UI	Not a first-class module (seed/DB for users)

End of FSD. Requirements and acceptance criteria are in docs/FRD.md.